Direct PIO/GPIO Address Lookup on Linux

Reverse Engineering Method for Cyclone V SoC FPGA Peripherals Without the Qsys File

Objective: When taking over a SoC FPGA board (Cyclone V) running Linux without having the hardware configuration file (Qsys/Platform Designer) available, we can use the Linux system itself to determine the physical addresses where the peripherals (PIO/GPIO) are located, and then verify them through direct hardware testing.

Step 1: Extract the Running Device Tree (Runtime DTB) into a Text File

The Linux operating system always keeps a copy of the currently running hardware configuration in memory. We will export this configuration as a source-format text file (`.dts`) for easier lookup.

```
dtc -I fs -O dts /sys/firmware/devicetree/base -o running.dts
```

This command reads the hardware information from `/sys/firmware/devicetree/base` and decompiles it into the `running.dts` file. At this point, this is the only source of truth.

Step 2: Locate PIO/GPIO Nodes

Instead of searching by device name (because we do not know what name the hardware designer assigned to the device), we search by the device driver's "family".

```
grep -B 15 "compatible = \"altr,pio" running.dts | grep -E "^\s*\S+@|reg = "
```

Result explanation: This command finds all blocks using the `altr,pio` driver (the standard Altera/Intel driver for PIO) and prints their node names and relative addresses. The result typically looks like:

```
gpio@0x100000180 {
    reg = <0x1 0x180 0x10>;
}
```

Analysis: `reg = <0x1 0x180 0x10>` means that this block is located in memory space number `1` (`0x1`), at offset `0x180` , with a size of `0x10` bytes.

Step 3: Find the Address Mapping Rule (`ranges`) to Calculate the Physical Address

The offset `0x180` cannot be used directly yet. We need to know how the bridge connecting the HPS (ARM) and FPGA translates this memory space into the actual physical address.

```
grep -B 5 "ranges = <0x1" running.dts | grep -E "ranges ="
```

Result explanation: This command searches for the address translation table. The result will look like:

```
ranges = <0x1 0x180 0xff200180 0x10 0x1 0x190 0xff200190 0x10 ...>;
```

How to read it: Each group consists of four parameters (subspace ID, Offset, Physical Address, Size). Looking at the first group, we can see that offset `0x180` is directly mapped to physical address `0xff200180`.

Quick tip for Cyclone V: The Lightweight HPS-to-FPGA Bridge is always assigned a fixed base address of `0xFF200000`. Therefore, if the peripheral offset is `0x180`, the physical address is definitely `0xFF200180`. Similarly, offset `0x190` corresponds to `0xFF200190`.

Step 4: Experimental Interaction (Blind Testing) Using `devmem`

At this point, we have a list of physical addresses (e.g. `0xFF200180`, `0xFF200190`, `0xFF2001A0`), but we do not know exactly which one is the LED, Button, or Switch. We must use the `devmem` tool to interact directly with the hardware and observe the results.

Test 1: Find an Output (Example: LED)

Write a random value to the address we just found:

```
devmem 0xFF200180 32 0x3FF
```

If the LEDs on the board light up, we can confidently record:

`0xFF200180` is the LED control block.

To turn it off:

```
devmem 0xFF200180 32 0x000
```

Test 2: Find an Input (Example: Push Button / Switch)

For the remaining addresses, continuously read their values while physically interacting with the board (toggle the switch or press the button):

```
devmem 0xFF200190 32
```

If the returned value displayed on the console changes immediately when you toggle the switch, we record:

`0xFF200190` is the Switch/Button reading block.

Operation Flow Summary (Cheat Sheet)

Copy and run the following commands sequentially for a quick lookup when working with a new board:

```
# 1. Extract the currently running hardware configuration
dtc -I fs -O dts /sys/firmware/devicetree/base -o running.dts

# 2. Scan all IPs belonging to the PIO/GPIO type to obtain their offsets
grep -B 15 "compatible = \"altr,pio" running.dts | grep -E "^\s*\S+@|reg = "

# 3. Scan the Bridge address mapping table to calculate the physical
addresses
grep "ranges = <0x1" running.dts

# 4. Verify directly using hardware testing
#    (replace 0xFF200180 with the calculated physical address)
devmem 0xFF200180 32
```